

DocBook Islamabad

Project Guide & Developer Notes

How This Advanced Website Was Built — Step by Step

Muhammad Raees | May 2026

1. Project Overview

DocBook Islamabad is a real-time doctor booking and discovery platform for Islamabad, Pakistan. It aggregates live doctor data from 6+ healthcare websites, shows searchable profiles, and allows patients to book appointments — all without a traditional backend server.

Live URL:

<https://raeesmalik89-oss.github.io/IslamabadDoctors/>

Feature	Description
Pages	Home, Find Doctors, Community News, Register Practice
Doctor Data	Auto-updated every 6 hours from live sources
Booking	Patients book via form — email sent instantly
Registration	Doctors register via form — email sent to admin
Community	AI-filtered live Islamabad news from 6 sources
SEO	Meta tags, sitemap, Google Search Console verified
Hosting	GitHub Pages — 100% free, no server needed

2. Tech Stack Used

This website uses NO paid hosting and NO backend server. Here is every technology used:

Technology	What It Does
HTML5 / CSS3	All page structure and design — written by hand
JavaScript (ES6+)	Makes the site interactive — search, filters, modals, booking
Python 3 + Playwright	Scrapes doctor data from Marham, Oladoc, InstaCare etc.
GitHub Actions	Runs the scraper automatically every 6 hours on GitHub servers
GitHub Pages	Hosts the website for free directly from the GitHub repository

Technology	What It Does
doctors.json	A JSON file that stores all doctor data — updated by the scraper
Formspree API	Handles form submissions and sends email notifications — free tier
rss2json.com API	Converts RSS news feeds into JSON that JavaScript can read
Google Search Console	Tells Google to index the website for search results

3. How the Architecture Works

Understanding the architecture is the key to building advanced websites like this. Most beginners think you need a database and a server. This site proves you do not.

3.1 The Data Flow

Here is exactly how doctor data gets from the source websites to a patient's screen:

Step-by-Step Data Flow

- Step 1: GitHub Actions triggers scraper.py every 6 hours (cron schedule)
- Step 2: scraper.py uses Playwright to open Marham, Oladoc, InstaCare etc. like a browser
- Step 3: It extracts doctor data (name, specialty, area, fee, rating) from each page
- Step 4: It saves all doctors into a single file called doctors.json
- Step 5: GitHub Actions commits and pushes doctors.json back to the repository
- Step 6: When a patient visits the site, app.js fetches doctors.json from GitHub
- Step 7: JavaScript renders the doctor cards on the page in real-time

3.2 Why No Backend Server?

Traditional websites need a server (Node.js, PHP, Django) running 24/7. This site replaces the server with two free services:

- GitHub Actions acts as the scheduled task runner (like a cron job on a server)
- GitHub Pages serves the static HTML/CSS/JS files to visitors for free
- doctors.json is a flat-file database — no MySQL, no MongoDB needed
- Formspree is the email sender — no sendmail or SMTP server needed

This architecture is called a JAMstack (JavaScript, APIs, Markup). It is faster, cheaper, and easier to maintain than a traditional server-based site.

4. Every File Explained

File	Purpose
index.html	Homepage — hero section, search form, featured doctors, about section
doctors.html	Doctor search page — filters, listing cards, booking modal
register-doctor.html	Doctor registration form — sends data to admin via Formspree
community.html	Live Islamabad news page — fetches RSS feeds, AI filters for relevance
app.js	Main JavaScript — loads doctors.json, handles search/filter, booking form
data.js	Fallback hardcoded doctor data — shown if doctors.json fails to load
style.css	All CSS styles — colors, layouts, responsive design, animations
scraper.py	Python scraper — uses Playwright to collect doctor data from 6 sources
doctors.json	Live doctor database — auto-updated every 6 hours by the scraper
requirements.txt	Python packages needed to run scraper.py
sitemap.xml	Tells Google which pages to index and how often they change
.gitignore	Tells Git to ignore temporary files like Word docs and Python cache
.github/workflows/scrape.yml	GitHub Actions workflow — schedules and runs the scraper automatically

5. How the Scraper Works

The scraper (scraper.py) is the most advanced part of this project. It uses Playwright — a headless browser — to visit websites just like a human would, then extracts data.

5.1 Sources Scraped

- **Uses __NEXT_DATA__ JSON embedded in Next.js pages** Marham.pk
- **Uses __NEXT_DATA__ JSON embedded in Next.js pages** Oladoc.com
- **Playwright headless browser scraping** InstaCare.pk
- **Playwright headless browser** Shifa International Hospital
- **HTTP requests + BeautifulSoup HTML parser** Kulsum International Hospital
- **Playwright headless browser** AIH (Advanced International Hospital)
- **30 manually verified Islamabad doctors always in the database** Hardcoded Doctors

5.2 What is Playwright?

Playwright is a Python library that controls a real Chrome browser automatically. It can click buttons, scroll pages, fill forms, and read the content — exactly like a human. This is needed because modern websites like Marham load their data using JavaScript, which normal HTTP requests cannot see.

5.3 What is __NEXT_DATA__?

Most modern websites (Marham, Oladoc) are built with Next.js, a JavaScript framework. Next.js embeds all the page data inside a hidden JSON block in the HTML:

```
<script id="__NEXT_DATA__"
  type="application/json">{"props":{"doctors":[...]}}</script>
```

The scraper finds this JSON and extracts it directly — much faster than parsing HTML. This is why these scrapers are reliable even when the page design changes.

5.4 GitHub Actions Automation

The file .github/workflows/scrape.yml runs the scraper automatically. The key line is:

```
cron: '0 */6 * * *' # Runs every 6 hours
```

After running, it commits doctors.json automatically:

```
git config user.email 'actions@github.com'
git add doctors.json
git commit -m 'Auto-update doctors data'
git push
```

6. How the Booking System Works

There is no server or database storing bookings. Here is the full flow:

Booking Flow

1. Patient clicks 'Book Appointment' on a doctor card
2. A modal popup appears with the booking form (patient name, phone, date, time, reason)
3. JavaScript fills hidden fields with the doctor's name and specialty
4. When patient submits, JavaScript sends a POST request to Formspree API
5. Formspree forwards all form data to raees.malik89@gmail.com instantly
6. The modal shows a success message to the patient
7. Admin (you) reads the email and contacts the patient on WhatsApp

Key JavaScript Code

```
const BOOKING_ENDPOINT = 'https://formspree.io/f/xqejlepb';
const res = await fetch(BOOKING_ENDPOINT, {
  method: 'POST', body: formData,
  headers: { 'Accept': 'application/json' }
});
```

The same Formspree endpoint handles both appointment bookings AND doctor registrations. Formspree's free tier allows 50 submissions per month. You can create a separate form ID for each if needed.

7. How the Community News Page Works

The community page (community.html) shows live Islamabad news without any backend. It uses a technique called client-side RSS fetching.

7.1 What is RSS?

RSS (Really Simple Syndication) is a standard format used by all major news websites to publish their latest articles. Dawn, Geo, ARY, Tribune all have RSS feeds. Example:

<https://www.dawn.com/feeds/home>

7.2 The rss2json.com Trick

Browsers cannot directly fetch RSS feeds from other websites (blocked by CORS security). The solution is rss2json.com — a free API that converts any RSS feed to JSON:

https://api.rss2json.com/v1/api.json?rss_url=https://www.dawn.com/feeds/home

JavaScript calls this API and gets back clean JSON with article titles, dates, descriptions and links.

7.3 AI Keyword Scoring

Not every article from Dawn is about Islamabad. The page uses a keyword scoring system to filter relevance:

- Islamabad / Rawalpindi / CDA / ICT mentions → +5 points each
- Health / hospital / doctor / PMDC keywords → +4 points each
- Events / weather / traffic keywords → +4 points each
- Government / parliament keywords → +3 points each

Articles scoring above the threshold are shown. Lower-scoring articles are hidden. This is a simple form of AI relevance filtering without any machine learning model.

8. SEO — How to Get Found on Google

8.1 What Was Added

Every page now has these SEO elements in the <head> section:

- **Short summary Google shows in search results** meta name='description'
- **Target search phrases like 'doctor islamabad'** meta name='keywords'
- **Title shown when link shared on WhatsApp/Facebook** meta property='og:title'
- **Description shown in WhatsApp/Facebook previews** meta property='og:description'
- **Tells Google the official URL to avoid duplicate content** link rel='canonical'
- **Lists all pages so Google can find them automatically** sitemap.xml

8.2 Google Search Console

After adding SEO tags, the site was verified on Google Search Console by:

- Creating a HTML verification file: googleca0518415b2fcbac.html
- Uploading it to the GitHub repository
- Clicking Verify on Google Search Console
- Submitting sitemap.xml to tell Google about all pages

Google will now crawl and index the site. Expect to appear in search results within 2-4 weeks for keywords like 'book doctor islamabad' and 'find doctor islamabad'.

9. Concept Guide — How to Build Advanced Websites Yourself

Here is everything you need to know to build sites like this from scratch. Follow this learning path:

Step 1: Learn HTML & CSS (2-3 weeks)

HTML is the structure (headings, paragraphs, forms, buttons). CSS is the design (colors, fonts, layouts, spacing). Every website starts here.

- Learn: div, p, h1-h6, a, img, form, input, button, ul, li
- Learn: flexbox and grid for layouts
- Learn: media queries for mobile-responsive design
- Practice: Build a personal portfolio page

Step 2: Learn JavaScript (3-4 weeks)

JavaScript makes websites interactive. It runs in the browser.

- Learn: variables, functions, if/else, loops, arrays, objects
- Learn: DOM manipulation (document.getElementById, addEventListener)
- Learn: fetch() API for calling external APIs
- Learn: async/await for handling responses
- Practice: Build a weather app using a free weather API

Step 3: Learn Git & GitHub (1 week)

Git tracks your code changes. GitHub stores your code online and hosts static sites for free.

- Learn: git init, git add, git commit, git push
- Learn: branches — master/main
- Learn: GitHub Pages for free hosting
- Practice: Push your portfolio to GitHub and make it live

Step 4: Learn Python for Automation (3-4 weeks)

Python is used for scraping data, automating tasks, and processing files.

- Learn: variables, functions, loops, lists, dictionaries
- Learn: requests library for HTTP calls

- Learn: BeautifulSoup for parsing HTML
- Learn: json library for reading/writing JSON files
- Learn: Playwright for headless browser automation
- Practice: Build a scraper that collects product prices from a website

Step 5: Learn GitHub Actions (1 week)

GitHub Actions lets you run code automatically on a schedule — for free.

- Learn: YAML syntax (indentation matters!)
- Learn: cron expressions for scheduling (0 */6 * * * = every 6 hours)
- Learn: how to install dependencies and run Python scripts in Actions
- Practice: Set up a workflow that runs a Python script every day

Step 6: Learn APIs (2 weeks)

APIs (Application Programming Interfaces) let your website talk to other services.

- Learn: REST APIs — making GET and POST requests
- Learn: JSON format — how data is structured
- Learn: Free APIs: Formspree (forms), rss2json (news), OpenWeather (weather)
- Practice: Build a page that fetches and displays live news using rss2json

Step 7: Learn SEO Basics (1 week)

- Add meta description and keywords to every page
- Use descriptive page titles with target keywords
- Create a sitemap.xml and submit to Google Search Console
- Make your site fast — compress images, minimize CSS/JS
- Get backlinks by sharing on social media and WhatsApp groups

10. Key Lessons Learned on This Project

Most Important Lessons

1. You do NOT need a server to build powerful websites
2. GitHub Actions is free and can automate almost any task
3. JSON files can replace a database for many use cases
4. Free APIs (Formspree, rss2json) replace entire backend systems
5. Playwright can scrape any modern website including JavaScript-heavy ones
6. `__NEXT_DATA__` is a fast and reliable way to extract data from Next.js sites
7. Always add `.gitignore` to avoid committing personal files accidentally
8. SEO meta tags and Google Search Console are essential for organic traffic
9. WhatsApp groups are the fastest way to get initial traffic in Pakistan

10. Client-side RSS fetching with keyword scoring = simple AI news filtering

11. What to Build Next (Upgrade Ideas)

Feature	How to Build It
Patient login & history	Add Firebase Authentication — free for small apps
Real-time chat with doctor	Use Firebase Firestore real-time database
Doctor dashboard	Password-protected page using Firebase Auth + Firestore
Payment for Featured listings	Integrate JazzCash or EasyPaisa API
Mobile app	Convert site to PWA (Progressive Web App) — no app store needed
Telemedicine / video calls	Integrate Daily.co or Agora free video API
SMS appointment reminders	Use Twilio SMS API (free trial available)
Doctor reviews & ratings	Store in Firebase Firestore, display on profile
Multi-city expansion	Add Lahore, Karachi with separate scrapers
Custom domain	Buy docbookpk.com (~\$10/yr) and point to GitHub Pages

12. Contact & Repository

Item	Details
Developer	Muhammad Raees
Email	raees.malik89@gmail.com
WhatsApp	+92-316-881-8693
GitHub	github.com/raeesmalik89-oss
Live Site	raeesmalik89-oss.github.io/IslamabadDoctors/
Repository	github.com/raeesmalik89-oss/IslamabadDoctors

Built with dedication for Islamabad — making quality healthcare accessible to everyone.